



Training Python-Django - 0

Starting

*Summary: Today, we'll embark on a journey to discover the basics of the syntax and semantics of **Python**.*

Version: 2

Contents

I	Preamble	2
II	General rules	3
III	Today's Specific Rules	4
IV	Exercise 00 : my first variables	5
V	Exercise 01 : Numbers	6
VI	Exercise 02: My first dictionary	7
VII	Exercise 03: Key search	9
VIII	Exercise 04: Search by value	10
IX	Exercise 05: Search by key or value	11
X	Exercise 06: Dictionary sorting	12
XI	Exercise 07: Periodic table of the elements	13
XII	Submission and Peer Evaluation	15

Chapter I

Preamble

The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one—and preferably only one—obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea—let's do more of those!



`import this`

Chapter II

General rules

- Your project must be realized in a virtual machine.
- Your virtual machine must have all the necessary software to complete your project. These softwares must be configured and installed.
- You can choose the operating system to use for your virtual machine.
- You must be able to use your virtual machine from a cluster computer.
- You must use a shared folder between your virtual machine and your host machine.
- During your evaluations you will use this folder to share with your repository.
- Your functions should not quit unexpectedly (segmentation fault, bus error, double free, etc) apart from undefined behaviors. If this happens, your project will be considered non functional and will receive a 0 during the evaluation.
- We encourage you to create test programs for your project even though this work **won't have to be submitted and won't be graded**. It will give you a chance to easily test your work and your peers' work. You will find those tests especially useful during your defence. Indeed, during defence, you are free to use your tests and/or the tests of the peer you are evaluating.
- Submit your work to your assigned git repository. Only the work in the git repository will be graded. If Deepthought is assigned to grade your work, it will be done after your peer-evaluations. If an error happens in any section of your work during Deepthought's grading, the evaluation will stop.

Chapter III

Today's Specific Rules

- No code in the global scope. We want functions.
- Each submitted file must end with a function call in a condition identical to:

```
if __name__ == '__main__':  
    your_function(whatever, parameters, are, required)
```

- You may implement error handling in this condition.
- No imports are allowed, except those explicitly mentioned in the "Authorized Functions" section of each exercise.
- You do not need to handle exceptions raised by the `open` function.
- You must use the `python3` interpreter.

Chapter IV

Exercise 00 : my first variables

	Exercise 00
Exercise 00 : my first variables	
Turn-in directory : <i>ex00/</i>	
Files to turn in : var.py	
Allowed functions : n/a	

Create a script named `var.py` in which you will define a `my_var` function. In this function, you must declare nine variables of different types and print them to the standard output. You must reproduce the following output exactly:

```
$> python3 var.py
42 has a type <class 'int'>
42 has a type <class 'str'>
quarante-deux has a type <class 'str'>
42.0 has a type <class 'float'>
True has a type <class 'bool'>
[42] has a type <class 'list'>
{42: 42} has a type <class 'dict'>
(42,) has a type <class 'tuple'>
set() has a type <class 'set'>
$>
```

Of course, explicitly writing the types of your variables in your print statements is strictly **prohibited**. Don't forget to call your function at the end of your script, as required by the instructions:

```
if __name__ == '__main__':
    my_var()
```

Chapter V

Exercise 01 : Numbers

	Exercise 01
Exercise 01 : Numbers	
Turn-in directory : <i>ex01/</i>	
Files to turn in : numbers.py	
Allowed functions : n/a	

For this exercise, you are free to define as many functions as you like and to name them as you wish.

The `d01.tar.gz` tarball, provided in the appendix of this subject, contains an `ex01/` subfolder that includes a `numbers.txt` file containing the numbers from 1 to 100, separated by commas.

Design a Python script named `numbers.py` whose role is to open the `numbers.txt` file, read the numbers it contains, and display them on the standard output, one per line, without any commas.

Chapter VI

Exercise 02: My first dictionary

	Exercise 02
Exercise 02: My first dictionary	
Turn-in directory : <i>ex02/</i>	
Files to turn in : <code>var_to_dict.py</code>	
Allowed functions : N.A.	

Once again, you are free to define as many functions as you like and to name them as you wish. This instruction will not be repeated unless explicitly stated otherwise.

Create a script named `var_to_dict.py` in which you will copy the following list of (key, value) pairs into one of your functions:

```
d = [
    ('Hendrix' , '1942'),
    ('Allman' , '1946'),
    ('King' , '1925'),
    ('Clapton' , '1945'),
    ('Johnson' , '1911'),
    ('Berry' , '1926'),
    ('Vaughan' , '1954'),
    ('Cooder' , '1947'),
    ('Page' , '1944'),
    ('Richards' , '1943'),
    ('Hammett' , '1962'),
    ('Cobain' , '1967'),
    ('Garcia' , '1942'),
    ('Beck' , '1944'),
    ('Santana' , '1947'),
    ('Ramone' , '1948'),
    ('White' , '1975'),
    ('Frusciante' , '1970'),
    ('Thompson' , '1949'),
    ('Burton' , '1939')
]
```

Your script must transform this variable into a dictionary. The year will be the key, and the name of the musician the value. It must then display this dictionary on the standard output using a clear format:

```
1970 : Frusciante  
1954 : Vaughan  
1948 : Ramone  
1944 : Page Beck  
1911 : Johnson  
...
```



The final order does not have to match the example. This is normal behaviour. Do you know why?

Chapter VII

Exercise 03: Key search

	Exercise 03
Exercise 03: Key search	
Turn-in directory : <i>ex03/</i>	
Files to turn in : <code>capital_city.py</code>	
Allowed functions : <code>import sys</code>	

Here are dictionaries that you must copy unaltered into one of your script's functions:

```
states = {
    "Oregon" : "OR",
    "Alabama" : "AL",
    "New Jersey": "NJ",
    "Colorado" : "CO"
}

capital_cities = {
    "OR": "Salem",
    "AL": "Montgomery",
    "NJ": "Trenton",
    "CO": "Denver"
}
```

Write a program that takes a state as an argument (e.g. Oregon) and displays its capital city (e.g. Salem) on the standard output. If the argument does not match any state, your script must display: **Unknown state**. If there is no argument, or more than one, your script must not do anything and should exit.

```
$> python3 capital_city.py Oregon
Salem
$> python3 capital_city.py Ile-De-France
Unknown state
$> python3 capital_city.py
$> python3 capital_city.py Oregon Alabama
$> python3 capital_city.py Oregon Alabama Ile-De-France
$>
```

Chapter VIII

Exercise 04: Search by value

	Exercise 04
Exercise 04: Search by value	
Turn-in directory : <i>ex04/</i>	
Files to turn in : state.py	
Allowed functions : import sys	

You are given the same dictionaries as in the previous exercise. You must copy them unaltered into one of your script's functions.

Create a program that takes a capital city as an argument and displays the corresponding state. The rest of your program's behaviour must remain the same as in the previous exercise.

```
$> python3 state.py Salem
Oregon
$> python3 state.py Paris
Unknown capital city
$> python3 state.py
$>
```

Chapter IX

Exercise 05: Search by key or value

	Exercise 05
Exercise 05: Search by key or value	
Turn-in directory : <i>ex05/</i>	
Files to turn in : all_in.py	
Allowed functions : import sys	

Starting with the same dictionaries, you must copy them unaltered again into one of your script functions, and write a program that behaves as follows:

- The program must take as argument a string containing as many expressions as are searched for, separated by commas.
- For each expression in this string, the program must detect whether it is a capital, a state, or neither.
- The program must be case-insensitive. It must also ignore multiple spaces.
- If there is no parameter or more than one parameter, the program must not display anything.
- When there are two successive commas in the string, the program must not display anything.
- The program must display results separated by a newline and strictly use the following format:

```
$> python3 all_in.py "New jersey, Tren ton, NewJersey, Trenton, toto, , sAlem"
Trenton is the capital of New Jersey
Tren ton is neither a capital city nor a state
NewJersey is neither a capital city nor a state
Trenton is the capital of New Jersey
toto is neither a capital city nor a state
Salem is the capital of Oregon
$>
```

Chapter X

Exercise 06: Dictionary sorting

	Exercise 06
Exercise 06: Dictionary sorting	
Turn-in directory : <i>ex06/</i>	
Files to turn in : <i>my_sort.py</i>	
Allowed functions : N.A.	

Integrate this dictionary into one of your functions as follows:

```
d = {
    'Hendrix' : '1942',
    'Allman' : '1946',
    'King' : '1925',
    'Clapton' : '1945',
    'Johnson' : '1911',
    'Berry' : '1926',
    'Vaughan' : '1954',
    'Cooder' : '1947',
    'Page' : '1944',
    'Richards' : '1943',
    'Hammett' : '1962',
    'Cobain' : '1967',
    'Garcia' : '1942',
    'Beck' : '1944',
    'Santana' : '1947',
    'Ramone' : '1948',
    'White' : '1975',
    'Frusciante' : '1970',
    'Thompson' : '1949',
    'Burton' : '1939',
}
```

Write a program that displays the names of the musicians sorted by year in ascending order, and alphabetically for identical years. Print one name per line, without displaying the year.

Chapter XI

Exercise 07: Periodic table of the elements

	Exercise 07
Exercise 07: Periodic table of the elements	
Turn-in directory : <code>ex07/</code>	
Files to turn in : <code>periodic_table.py</code>	
Allowed functions : <code>import sys</code>	

The `d01.tar.gz` tarball in the appendix of this subject contains the `ex07/` subfolder in which you will find a `periodic_table.txt` file that describes a periodic table of the elements in a format designed for programmers.

Create a program that uses this file to generate an HTML page representing the periodic table of the elements in a proper format.

- Each element must be displayed in a HTML table cell.
- The name of each element must appear in an `h4` heading tag.
- The attributes of each element must be displayed as a list. The list must include at least the atomic number, the symbol, and the atomic mass.
- You must respect the layout of the Mendeleiev periodic table as it appears on Google. Empty cells must be preserved where appropriate, as well as line breaks where necessary.

Your program must generate a file named `periodic_table.html`. This HTML file must be readable in any browser and must be W3C-valid.

You are free to design your program as you wish. Do not hesitate to split your code into reusable functions. You may customize the output using inline CSS styles to improve readability (for example, table border colors). You may also generate a

periodic_table.css file if you prefer.

Here is an excerpt of an example output to give you an idea:

```
[...]
<table>
  <tr>
    <td style="border: 1px solid black; padding:10px">
      <h4>Hydrogen</h4>
      <ul>
        <li>No 1</li>
        <li>H</li>
        <li>1.00794</li>
        <li>1 electron</li>
      </ul>
    </td>
  </tr>
[...]
```

Chapter XII

Submission and Peer Evaluation

Turn in your assignment in your `Git` repository as usual. Only the work contained in your repository will be evaluated during the defense. Do not hesitate to double-check the names of your folders and files to ensure they are correct.



The evaluation process will take place on the computer of the evaluated group.